COURSE PROGRESSION **Between stages 2 and 3**

1 Instructor-led practice

2 Supported practice

3 Independent practice (ahead)

Supported practice with growing independence. Instructor guidance continues; fully independent work comes later.

← Action 10A

Engine Trials / lecture outline

Recap → compare → test → explain → submit. Today's goal: justify one tool choice with one observed result.

Suggested pacing: 20 min recap; 20 min lecture; 25 min research; 50 min trial; 25 min OOP/AI/peer check; 20 min GitHub/receipt/submission. Leave room for the scheduled break. Adjust to the class.

Explain the change in responsibility

We are between supported practice and independent practice, with growing responsibility for our own work. We are not fully independent yet. The instructor continues to guide the class through lectures, checkpoints and feedback. Use the requirements, documentation, your team, AI and other tools to choose a next step, test it and ask focused questions. Fully independent work comes later; it still uses teams and tools rather than requiring you to work alone.

Professor prompt: "Show me your goal, the requirement, what you tried, what happened and your next question."

> **1. Recap first**
>
> Point from JSON/text input to validation, record objects, a list, the UI and saved output. Ask: which method changed the data?

> **2. Engine, editor, game**
>
> The engine supplies runtime systems. The editor arranges a project. A game is the project you make and run. raylib supplies code functions rather than a full editor.

> **3. Three hats**

Visual: camera, scene and feedback. Logical: input, conditions and methods. Data: position, score and state. Use one small object to connect them.

4. Supporting technology

SDK/compiler builds C#. Runtime executes it. GPU/graphics API draws. Physics tests collisions. Assets supply images, models and audio. Git records source changes.

5. OOP in a scene

A class describes a kind; an object is one instance. Fields/properties hold state; methods do work. Components add responsibilities; inheritance specializes a type. Engine APIs differ.

6. From events to a loop

WPF reacts to clicks. A game repeatedly reads input, updates state and draws a frame. Delta time helps movement track elapsed time rather than frame count.

7. Access and trade-offs

Compare open-source, proprietary and source-available terms separately from 2D/3D and C# support. Enterprise use is a scale/support context, not a mutually exclusive engine type. Portable does not guarantee no SDK or GPU requirements.

8. CLI and tooling

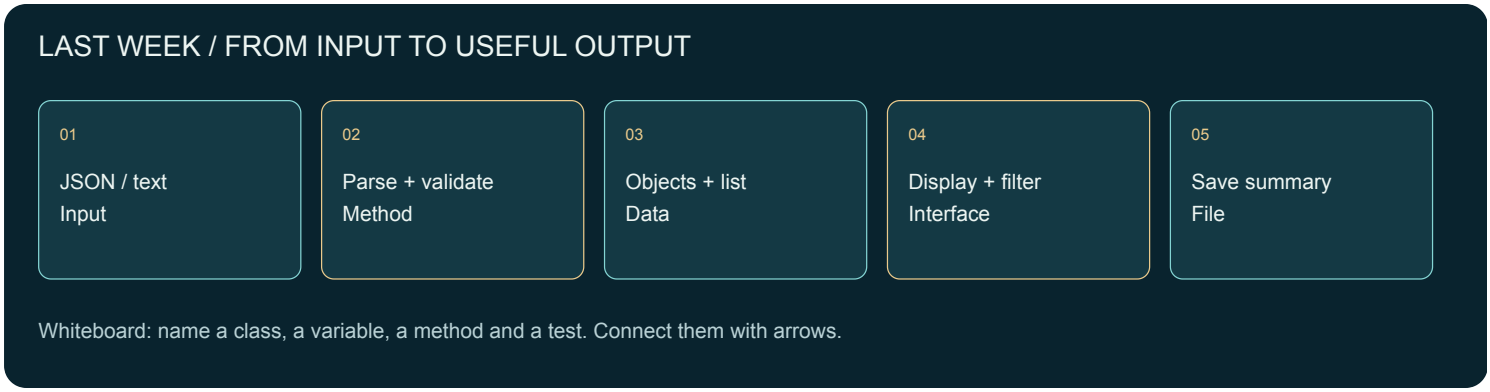
CLI means command-line interface. Unity editor batch-mode flags automate builds/tests; the CLI is not a separate engine. Use documented commands and existing access. Automation and AI do not replace checking the result.

9. Close the class

Peer-check the visible change and OOP map. Commit evidence and receipt; submit one GitHub URL. A discovery trial is enough today; a finished game is not required.

Whiteboard prompt

“Show how last week’s record object becomes this week’s game object. What is data, what is an action, and which system calls it?”



Finish with evidence

Ask a peer to point to the changed property and predict the next run. Check the receipt is inside GitHub before the Canvas URL is submitted.

Official references

- Godot .NET
- Godot .NET guide
- Stride 4.4
- Stride 4.4 guide
- Prowl
- Prowl guide
- Unity
- Unity guide
- Flax
- Flax guide
- raylib + raylib-cs
- raylib + raylib-cs guide
- Unreal Engine
- Unreal Engine guide
- GameMaker
- GameMaker guide
- RPG Maker MZ
- RPG Maker MZ guide
- Godot: nodes and scenes
- Godot: .NET and C#

- Unity: command-line arguments
- RPG Maker Unite: separate Unity-based product
- Free flowchart tool: diagrams.net

Source check: October 25, 2026. Licensing, versions and school access can change. Record the version used today; this class does not require purchases.